



ЦЕНТРАЛЬНЫЙ
УНИВЕРСИТЕТ

Оконные функции

SQL И БАЗЫ ДАННЫХ

НЕДЕЛЯ 6

Что такое оконная функция?

Оконная функция (ОФ) — это аналитическая функция в SQL, которая вычисляет значение (агрегат, ранг, сдвиг и т. д.) по группе строк (окну), связанной с текущей строкой через конструкцию **OVER (...)**, и возвращает результат в каждой строке без её группировки или удаления.



Синтаксис оконных функций

```
ОКОННАЯ_ФУНКЦИЯ ( [АРГУМЕНТЫ] ) OVER (
    [PARTITION BY ... , ... ]
    [ORDER BY ... , ... ]
    [ROWS | RANGES ... , ... ]
)
```

- **PARTITION BY** делит строки на окна (например, все строки с одинаковым user_id попадают в одно окно).
- **ORDER BY** определяет порядок строк внутри окна (например, по дате).
- **ROWS | RANGES...** определяет, какие именно строки брать (текущую, предыдущие, следующие).

Подробнее про окно

Окно — это подмножество строк, на котором для каждой текущей строки выполняется вычисление (агрегат, ранг, навигация).

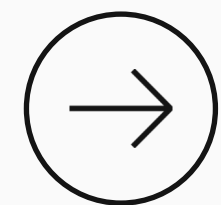
Задаётся с помощью ключевого слова **PARTITION BY**, которое разбивает весь набор строк на независимые группы — окна.

```
ОКОННАЯ_ФУНКЦИЯ ( [АРГУМЕНТЫ] ) OVER (
    [PARTITION BY ... , ... ]
)
```

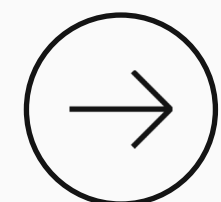
Ключевое слово **PARTITION BY** не является обязательным. Без него окном будет выступать весь набор строк.

Подробнее про сортировку

За сортировку отвечает ключевое слово **ORDER BY**. Оно выполняет две ключевые задачи.



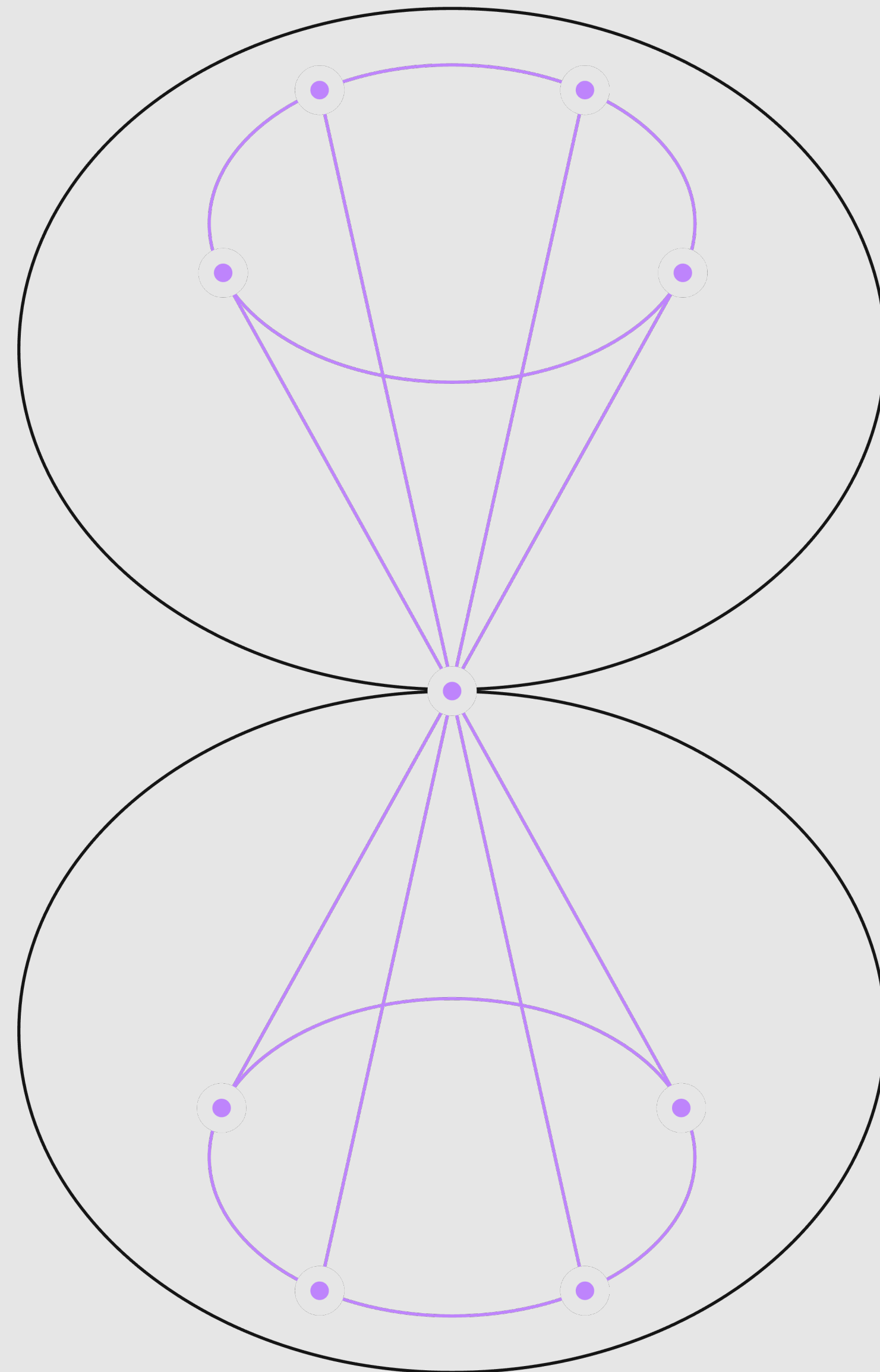
Определяет порядок строк внутри каждого окна.



Влияет на логику расчёта: какие именно «предыдущие/следующие» или «первые/последние» строки попадут в рамки.

```
ОКОННАЯ_ФУНКЦИЯ ( [АРГУМЕНТЫ] ) OVER (
    [PARTITION BY ... , ... ]
    [ORDER BY ... , ... ]
)
```

Агрегирующая оконная функция



Вид оконной функции, которая внутри каждого окна (набора строк) вычисляет агрегатное значение (сумму, среднее, максимум и т. д.) и возвращает результат для каждой строки этого окна, не изменяя самих строк и не объединяя их в одну.

COUNT

MIN

SUM

MAX

AVG

Синтаксис агрегирующей оконной функции

```
АГРЕГИРУЮЩАЯ_ОКОННАЯ_ФУНКЦИЯ ( [АРГУМЕНТЫ] ) OVER (  
    [PARTITION BY ... , ... ]  
)
```

PARTITION BY делит строки на окна.

РЕШАЕМ 1

Рассчитать среднюю стоимость покупок по пользователю, а также отклонение стоимости каждой покупки от среднего. Необходимо брать только завершённые заказы (`order_status = 'Completed'`).

Используй таблицу `shop_orders`.

Ответ должен содержать столбцы:

- `order_id` — идентификатор заказа;
- `customer_id` — идентификатор пользователя;
- `total_amount` — стоимость заказа;
- `avg` — средняя стоимость заказов по пользователю;
- `diff` — отклонение от средней стоимости заказов.

РЕШАЕМ 1

Рассчитать среднюю стоимость покупок по пользователю, а также отклонение стоимости каждой покупки от среднего.

```
SELECT
    order_id,
    customer_id,
    total_amount,
    avg(total_amount) OVER (
        PARTITION BY customer_id) AS avg,
    total_amount - avg(total_amount) OVER (
        PARTITION BY customer_id) AS diff
FROM shop_orders
WHERE order_status = 'Completed'
```

123 order_id ▼	123 customer_id ▼	123 amount ▼	123 avg ▼	123 diff ▼
1	111	1 000	37 000	-36 000
2	111	10 000	37 000	-27 000
3	111	100 000	37 000	63 000
4	222	1 000	500,5	499,5
5	222	1	500,5	-499,5
6	333	100	100	0

Агрегирующая оконная функция с нарастающим итогом

Оконная функция с нарастающим итогом отличается от агрегирующих оконных функций только наличием **ORDER BY**, которое упорядочивает окно по выбранному признаку. Обычно по дате, но возможны и другие варианты.

```
АГРЕГИРУЮЩАЯ_ОКОННАЯ_ФУНКЦИЯ ( [АРГУМЕНТЫ] ) OVER (
    [PARTITION BY ... , ... ]
    [ORDER BY ... , ... ]
)
```

РЕШАЕМ 2

Рассчитать выручку магазина накопительным итогом по месяцам за 2024 год. Накопительный итог суммирует выручку с первого месяца года до отчётного месяца. В расчётах должны участвовать только завершённые заказы (`order_status = 'Completed'`). Полученные результаты отсортируй по возрастанию отчётного месяца.

Ответ должен содержать столбцы:

- `month_dt` — отчётный месяц в формате даты;
- `revenue` — выручка за месяц;
- `cumulative_revenue` — выручка накопительным итогом с первого месяца года до отчётного месяца.

Используй таблицу `shop_orders`.

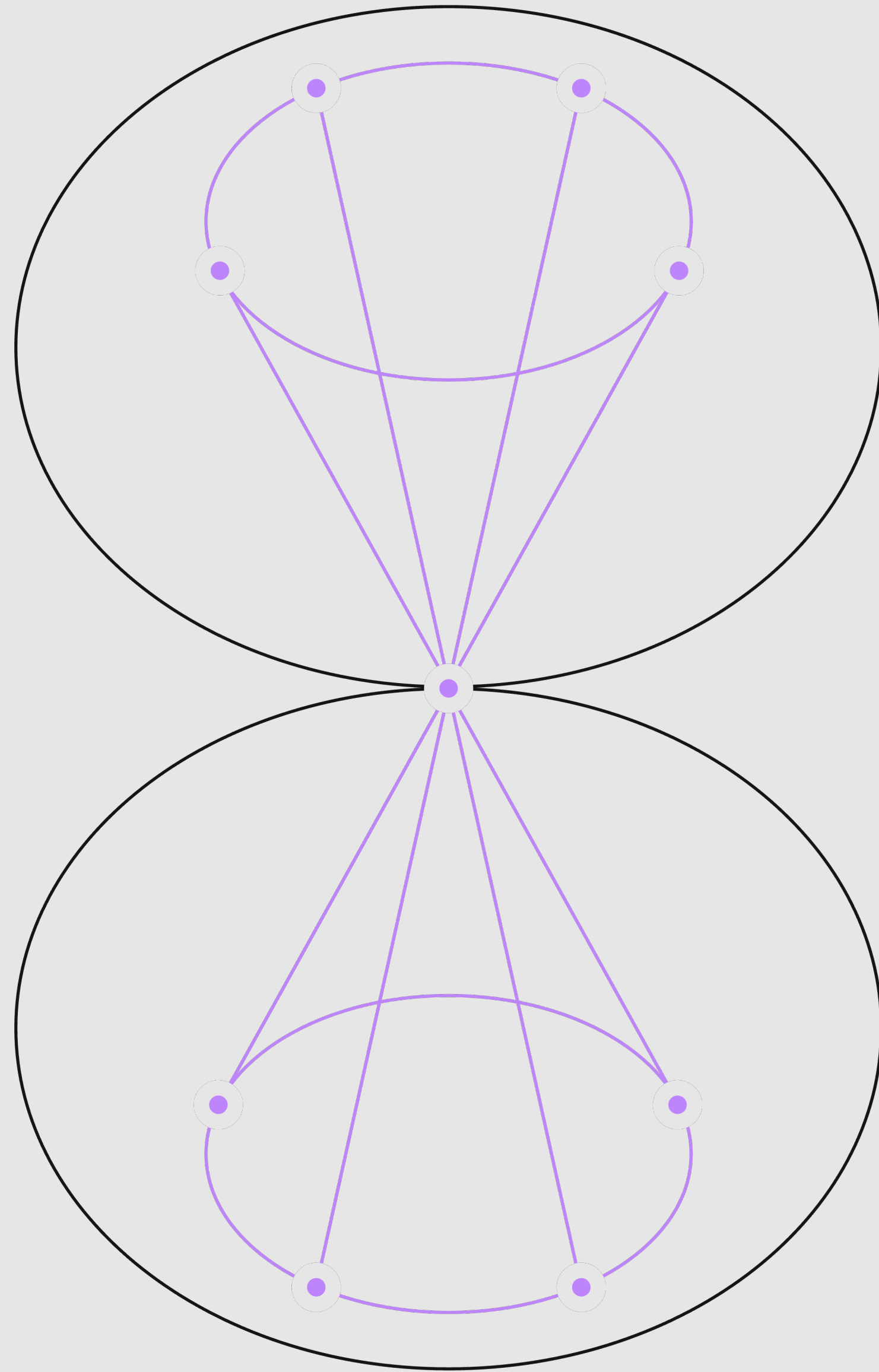
РЕШАЕМ 2

Рассчитать выручку магазина накопительным итогом по месяцам за 2024 год.

```
WITH revenue_tab AS (  
    SELECT  
        date_trunc('month', so.order_date)::date AS month_dt,  
        sum(so.total_amount) AS revenue  
    FROM shop_orders so  
    WHERE so.order_date BETWEEN date '2024-01-01'  
        AND date '2024-12-31'  
        AND so.order_status = 'Completed'  
    GROUP BY 1  
)  
SELECT  
    month_dt,  
    revenue,  
    sum(revenue) OVER (ORDER BY month_dt) AS  
cumulative_revenue  
FROM revenue_tab  
ORDER BY 1
```

	🕒 month_dt ▼	¹²³ revenue ▼	¹²³ cumulative_revenue ▼
1	2024-01-01	244 706 460	244 706 460
2	2024-02-01	247 684 170	492 390 630
3	2024-03-01	280 990 480	773 381 110
4	2024-04-01	301 547 320	1 074 928 430
5	2024-05-01	315 980 710	1 390 909 140
6	2024-06-01	331 188 970	1 722 098 110
7	2024-07-01	380 641 630	2 102 739 740
8	2024-08-01	423 283 700	2 526 023 440
9	2024-09-01	456 269 280	2 982 292 720
10	2024-10-01	537 461 350	3 519 754 070
11	2024-11-01	571 927 290	4 091 681 360

Ранжирующая оконная функция

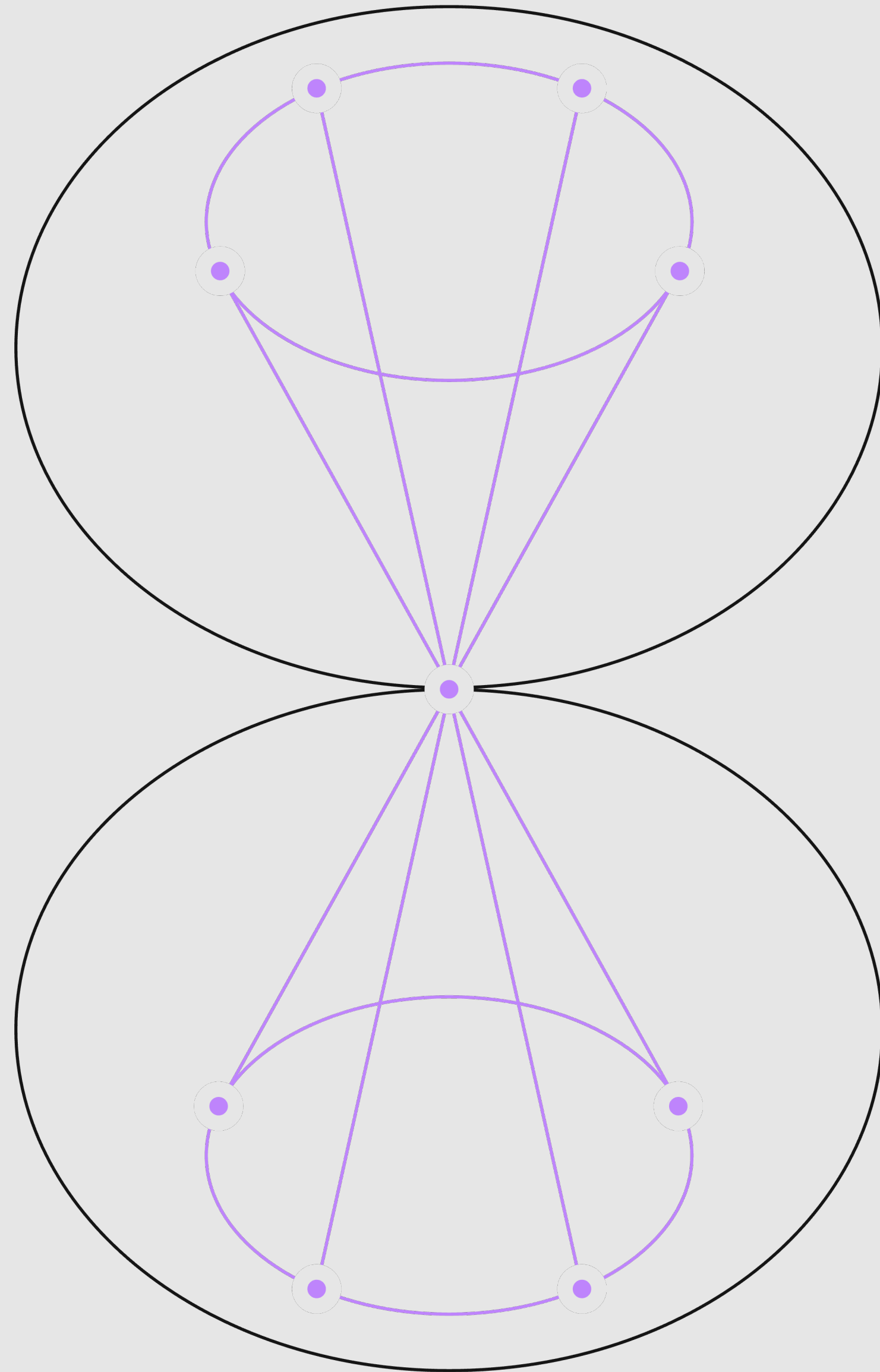


Ранжирующая оконная функция — это вид оконной функции, которая присваивает каждой строке внутри заданного окна (набора строк) порядковый номер или ранг в соответствии с заданным порядком сортировки.

Функции в SQL:

- `ROW_NUMBER,`
- `RANK,`
- `DENSE_RANK.`

Синтаксис ранжирующих оконных функций



```
РАНЖИРУЮЩАЯ_ОКОННАЯ_ФУНКЦИЯ()  
OVER (  
    [PARTITION BY ..., ... ]  
    [ORDER BY ..., ... ]  
)
```

- **PARTITION BY** делит строки на окна.
- **ORDER BY** определяет порядок строк внутри окна (например, по дате).

ROW_NUMBER()

Даёт уникальный последовательный номер каждой строке.

```
ROW_NUMBER() OVER (ORDER BY time) AS prize
```

users	time	prize
Галина	8.1	1
Анастасия	9.8	2
Фёдор	9.8	3
Алексей	10.1	4

Строкам с одинаковыми значения поля(ей) сортировки ранги присваиваются случайным образом.

RANK()

Присваивает одинаковый ранг строкам с равными значениями критерия сортировки.

Пропускает номера строк с повторяющимися местами в рейтинге.

```
RANK() OVER (ORDER BY time) AS prize
```

users	time	prize
Галина	8.1	1
Анастасия	9.8	2
Фёдор	9.8	2
Алексей	10.1	4

DENSE_RANK()

Похож на **RANK**, но без пропусков номеров после одинаковых значений.

```
DENSE_RANK() OVER (ORDER BY time)
```

users	time	prize
Галина	8.1	1
Анастасия	9.8	2
Фёдор	9.8	2
Алексей	10.1	3

Поиск лучших/первых/последних значений в группе

В бизнес-задачах часто требуется определить «лидеров» внутри группы объектов. Это могут быть как лучшие, так и худшие значения показателей — в зависимости от целей анализа.

Примеры:

- найти самый дорогой авиабилет, купленный в заданный период;
- определить товар с максимальными продажами внутри категории;
- выбрать фильм с наибольшим числом просмотров за неделю среди новинок.

Количество лидеров, которое нужно выделить, зависит от потребностей бизнеса: иногда ищут только первое место, а иногда — топ-3, топ-10 или даже «антилидеров» с наихудшими метриками.

РЕШАЕМ 3

Необходимо найти самый дорогой билет на каждый рейс.

Ответ должен содержать столбцы:

- flight_id,
- ticket_no,
- amount.

Используй таблицу fl_ticket_flights.

РЕШАЕМ 3

Необходимо найти самый дорогой билет на каждый рейс.

```
WITH rank_amount AS
(
    SELECT
        tf.flight_id,
        tf.ticket_no,
        tf.amount,
        ROW_NUMBER() OVER (
            PARTITION BY tf.flight_id
            ORDER BY tf.amount DESC
        ) AS rn
    FROM fl_ticket_flights AS tf
)
SELECT
    flight_id,
    ticket_no,
    amount
FROM rank_amount
WHERE rn = 1
```

Определение позиций в рейтинге

Часто нужно не просто найти лидера, а понять, какое место занимает объект среди других. Для этого используют ранжирующие функции, которые присваивают каждому элементу «позицию» в рейтинге.

Примеры:

- определить место рейса по длительности среди рейсов на том же маршруте,
- вычислить позицию клиента по сумме покупок внутри своего города,
- составить рейтинг фильмов по оценкам зрителей.

Такие задачи помогают увидеть не только абсолютные значения метрик, но и их относительное положение. Важно, что в зависимости от метода ранжирования при одинаковых показателях объекты могут занимать одно и то же место или получать разные последовательные номера.

РЕШАЕМ 4

Для аэропорта **Пулково** (LED, Санкт-Петербург) определить, какие направления прилета имеют самую короткую и самую длинную продолжительность полёта.

1. Выберите все уникальные пары аэропортов вылета и прилёта из Санкт-Петербурга (LED).
2. Рассчитайте плановую длительность каждого рейса.
3. Постройте рейтинг направлений по длительности перелёта, где 1 место — самый короткий рейс, а последнее место — самый долгий.

В результирующем наборе должны быть столбцы:

- `departure_airport` — аэропорт вылета (все значения должны быть LED);
- `arrival_airport` — аэропорт прибытия;
- `duration` — длительность перелёта по расписанию;
- `duration_rank` — позиция рейса в рейтинге по длительности.

РЕШАЕМ 4

ШАГ 1

1. Берём все рейсы, где аэропортом вылета является Пулково.
2. Рассчитываем плановую длительность каждого рейса.
3. Находим уникальные пары аэропортов вылета, прилёта и их плановой длительностью.

```
SELECT DISTINCT
    departure_airport,
    arrival_airport,
    scheduled_arrival - scheduled_departure as duration
FROM fl_flights
WHERE departure_airport = 'LED'
```

РЕШАЕМ 4

ШАГ 2

Проставляем рейтинг направлений по длительности перелёта в порядке возрастания

```
WITH tmp AS (  
    SELECT DISTINCT  
        departure_airport,  
        arrival_airport,  
        scheduled_arrival -  
        scheduled_departure  
    AS duration  
    FROM fl_flights  
    WHERE departure_airport = 'LED'  
)  
SELECT  
    *,  
    RANK() OVER (  
        ORDER BY duration)  
    AS duration_rank  
FROM tmp
```

	A-Z departure_airport	A-Z arrival_airport	duration	123 duration_rank
1	LED	SVO	00:50:00	1
2	LED	VKO	00:50:00	1
3	LED	DME	00:55:00	3
4	LED	GOJ	01:10:00	4
5	LED	CEE	01:25:00	5
6	LED	CSY	01:25:00	5
7	LED	KVX	01:30:00	7
8	LED	VOZ	01:30:00	7
9	LED	KZN	01:40:00	9

Функции LAG и LEAD

LAG и **LEAD** — это оконные функции смещения, которые позволяют «заглянуть» в предыдущую или следующую строки в рамках определённого окна.

Они помогают сравнивать значения между строками, не используя подзапросы или **JOIN**.
Параметры после **OVER** используются стандартные.

```
LAG(column[, offset] [, default]) OVER (...)  
LEAD(column[, offset] [, default]) OVER (...)
```

- **column** — из какой колонки взять значение.
- **offset** — сдвиг в строках (по умолчанию 1).
- **default** — что вернуть, если предыдущей или следующей строки нет (опционально).

РЕШАЕМ 5

Необходимо рассчитать время между завершёнными заказами (со статусом Completed) по каждому пользователю.

В результирующем наборе должны быть столбцы:

- `order_id` — идентификатор заказа;
- `customer_id` — идентификатор пользователя;
- `order_date` — дата заказа;
- `prev_order_date` — дата предыдущего заказа по пользователю;
- `diff_days` — количество дней между текущим и предыдущим заказом.

РЕШАЕМ 5

Необходимо рассчитать время между завершёнными заказами (со статусом Completed) по каждому пользователю.

```
SELECT
    order_id,
    customer_id,
    order_date,
    lag(order_date) OVER (
        PARTITION BY customer_id
ORDER BY order_date
    ) AS prev_order_date,
    order_date - lag(order_date)
OVER (
    PARTITION BY customer_id
ORDER BY order_date
    ) AS diff_days
FROM shop_orders
WHERE order_status = 'Completed'
```

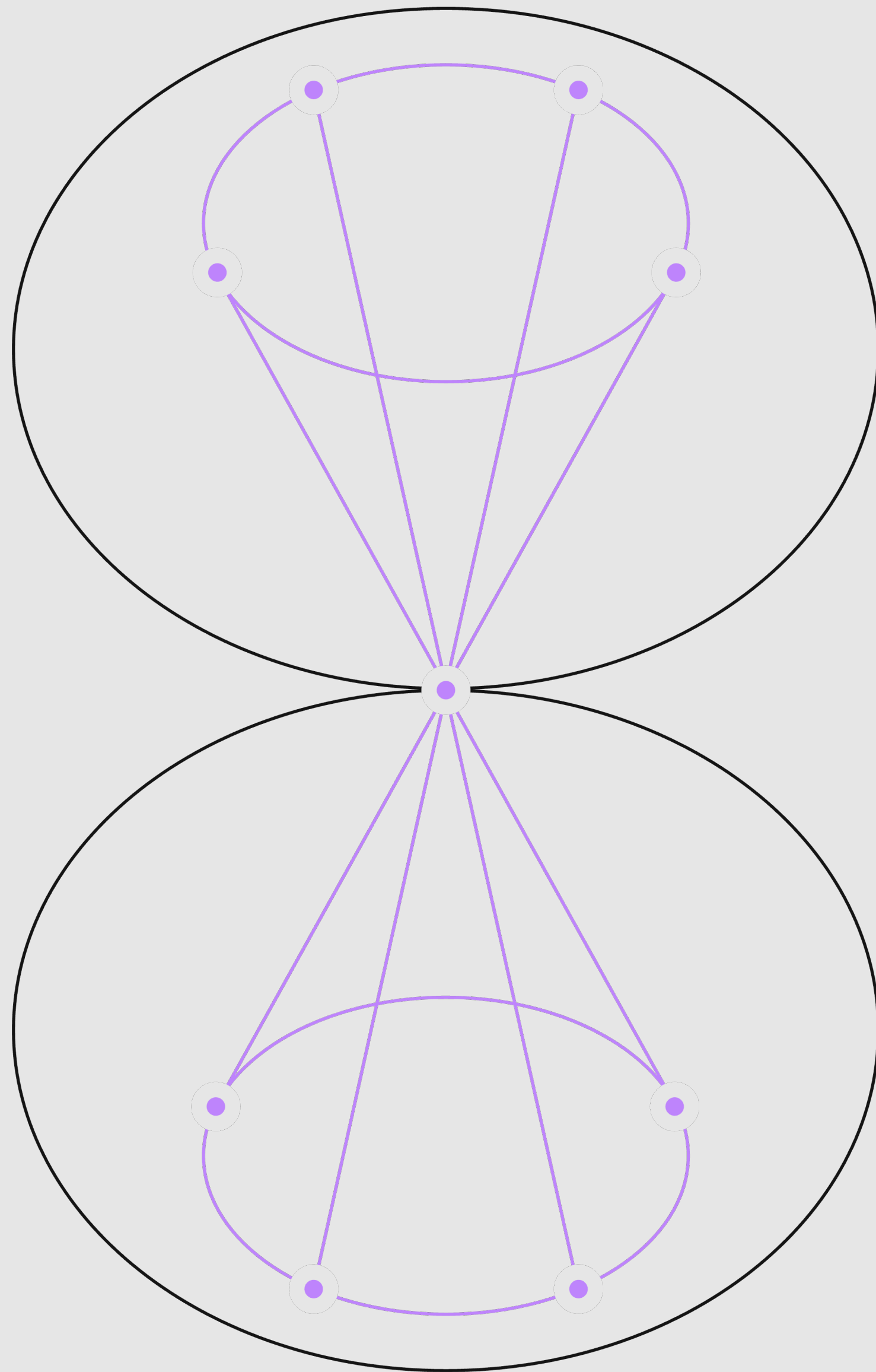
order_id	customer_id	order_date	prev_order_date	diff_days
16	10	2023-01-03	[NULL]	[NULL]
21	10	2023-02-28	2023-01-03	56
20	10	2023-07-18	2023-02-28	140
18	10	2024-04-01	2023-07-18	258
19	10	2024-08-29	2024-04-01	150
17	10	2024-11-27	2024-08-29	90

Установка границ в оконной функции

В оконной функции можно явным образом определить набор данных, который будет использоваться в расчётах для текущей строки.

Примеры задач:

- вычисления в скользящем окне,
- расчёт кумулятивных итогов,
- вычисления в «окрестности» значения.



Границы окна

Границы окна определяют тот набор строк в пределах окна (partition by), который будет передан в функцию для вычислений.

В SQL можно задать границы следующими способами.

Относительно текущей строки:

- [начало группы, текущая строка];
- [две предыдущие строки, текущая строка].

ROWS .. BETWEEN ..

Относительно текущего значения выражения сортировки:

- [прошлый день, день текущей строки];
- [начало группы, день текущей строки].

RANGE .. BETWEEN ..

Использование ROWS и RANGE

Границы окна можно задать в конце **OVER**.

```
ОКОННАЯ_ФУНКЦИЯ ( [АРГУМЕНТЫ] ) OVER (
    [PARTITION BY ...]
    [ORDER BY ...]
    [{ ROWS | RANGE } BETWEEN левая_граница AND правая_граница]
)
```

ROWS задаёт границы по позиции строк.

RANGE задаёт границы по значениям колонки(-ок) из **ORDER BY**.

Использование ROWS и RANGE

Граница	Описание
UNBOUNDED PRECEDING	От первой строки раздела
n PRECEDING	От n-й предыдущей строки или значения
CURRENT ROW	От/до текущей строки или значения
n FOLLOWING	До следующей n-й строки или значения
UNBOUNDED FOLLOWING	До последней строки раздела

* Раздел — группа строк partition by.

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

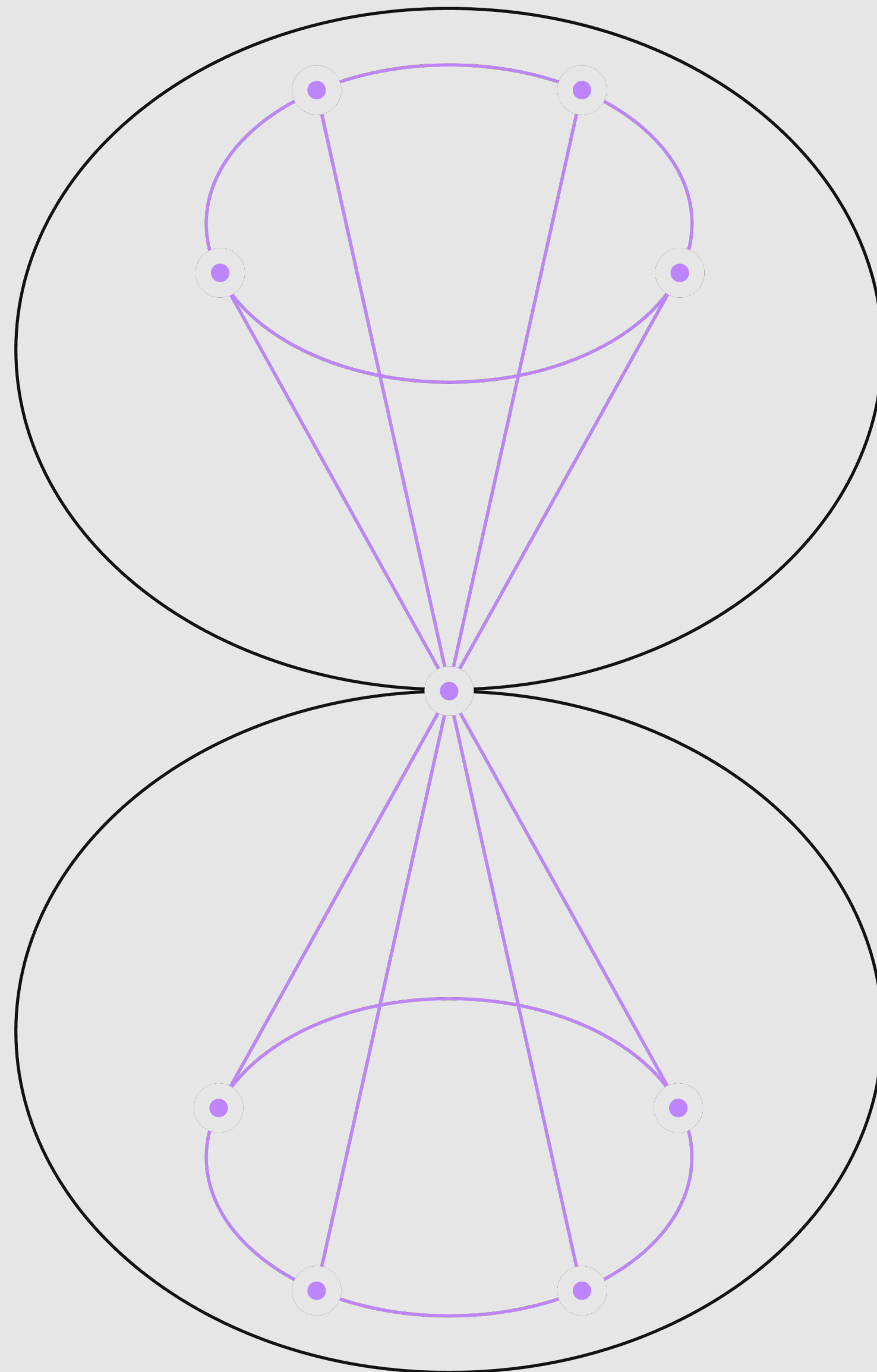
RANGE BETWEEN INTERVAL '7' DAY PRECEDING AND CURRENT ROW

Особенности ROWS и RANGES

ROWS выбирает строки независимо от повторяющихся значений **ORDER BY**.

В окне может оказаться меньше число строк.

RANGE объединяет все строки, имеющие одинаковое значение выражения **ORDER BY** и попадающие в диапазон (например, даты).



Границы по умолчанию

Ключевые слова для ручного изменения границ окна являются необязательными.
По умолчанию окно задаётся так.

- 01 Для агрегирующих оконных функций без сортировки: **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**.
- 02 Для агрегирующих оконных функций с сортировкой: **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**, т. е. «всё от начала раздела до текущей строки» по значению **ORDER BY**.
- 03 Для навигационных (**LAG**, **LEAD**) и ранжирующих (**ROW_NUMBER**, **RANK**) рамки не нужны: они оперируют всем разделом сразу.

Скольльзящие агрегаты

Иногда важно анализировать не всю группу целиком, а только «окно» вокруг текущего значения — несколько предыдущих или следующих строк, либо значения в определённом интервале времени.

Примеры:

- рассчитать среднюю длительность перелётов за последние 5 рейсов из аэропорта,
- посчитать сумму бронирований за последние 7 дней относительно каждой даты,
- определить максимальную задержку за три предыдущих рейса авиакомпании.

Подобные задачи помогают выявлять тренды, анализировать динамику и находить аномалии в поведении показателей.

РЕШАЕМ 6

Посчитать скользящую сумму заказов клиента за 3 последних заказа.

Ответ должен содержать следующие столбцы:

- order_id;
- customer_id;
- order_date;
- total_amount;
- rolling_sum_3 — сумма текущего и двух предыдущих заказов этого клиента (по дате).

РЕШАЕМ 6

Необходимо рассчитать время между завершёнными заказами (со статусом Completed) по каждому пользователю.

```
SELECT
    order_id,
    customer_id,
    order_date,
    total_amount,
    SUM(total_amount) OVER (
        PARTITION BY customer_id
        ORDER BY order_date
        ROWS BETWEEN 2
        PRECEDING AND CURRENT ROW
    ) AS rolling_sum_3
FROM shop_orders
```

123 order_id	123 customer_id	🕒 order_date	123 total_amount	123 rolling_sum_3
16	10	2023-01-03	49 990	49 990
21	10	2023-02-28	129 980	179 970
20	10	2023-07-18	75 990	255 960
18	10	2024-04-01	5 990	211 960
19	10	2024-08-29	29 990	111 970
17	10	2024-11-27	5 990	41 970

РЕШАЕМ 7

Для таблицы shop_orders:

- посчитать накопительную сумму
через ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW;
- посчитать
через RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.

Сравнить результат для даты '2023-01-11' и order_id < 20000.

РЕШАЕМ 7

Сравнение накопительных сумм

```
SELECT
    order_id,
    order_date,
    total_amount,
    -- ROWS
    SUM(total_amount) OVER (
        ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_rows,
    -- RANGE
    SUM(total_amount) OVER (
        ORDER BY order_date
        RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_range
FROM shop_orders
where order_date = '2023-01-11'
    and order_id < 20000
```


РЕШАЕМ 7

Сравнение накопительных сумм

¹²³ order_id ▼	 order_date ▼	¹²³ total_amount ▼	¹²³ running_rows ▼	¹²³ running_range ▼
470	2023-01-11	5 990	5 990	328 890
10 189	2023-01-11	39 990	45 980	328 890
5 962	2023-01-11	44 980	90 960	328 890
6 304	2023-01-11	49 990	140 950	328 890
7 671	2023-01-11	14 990	155 940	328 890
10 740	2023-01-11	39 990	195 930	328 890
349	2023-01-11	39 990	235 920	328 890
4 991	2023-01-11	39 990	275 910	328 890
5 098	2023-01-11	12 990	288 900	328 890
19 511	2023-01-11	39 990	328 890	328 890

Вопросы

